

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK

“RELASI ANTAR OBJEK
(KELAS DIAGRAM DAN ASOSIASI)”

disusun Oleh:
MUHAMMAD FEDORA ARGADYAKSA
2511533016

Dosen Pengampu:
Nurfiah, S.ST, M.Kom



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

1. Langkah Kerja Praktikum

- a. Langkah pertama yaitu, buka aplikasi yang dibutuhkan untuk praktikum (Eclipse) Lalu buat Java Project yang baru dan berisi Package Pekan 2.
- b. Langkah kedua, membuat kode program yang terdapat di dalam modul praktikum :
 - a. Class Rekening :

```
1. package Pekan_2;
2.
3. public class Rekening {
4.     String nomorRekening;
5.     String namaPemilik;
6.     double saldo;
7.
8.     public Rekening(String nomor, String nama, double saldoAwal) {
9.         nomorRekening = nomor;
10.        namaPemilik = nama;
11.        saldo = saldoAwal;
12.        System.out.println("Rekening atas nama " + namaPemilik +
" berhasil dibuat dengan saldo Rp " + saldo);
13.    }
14.
15.    public void setorTunai(double nominal) {
16.        if (nominal > 0){
17.            saldo += nominal;
18.            System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini : Rp" + saldo);
19.        } else {
20.            System.out.println("Gagal : Nominal setor harus
lebih dari 0!");
21.        }
22.    }
23.
24.    public void tarikTunai(double nominal) {
25.        if (nominal <= 10000 && nominal <= saldo) {
26.            saldo -= nominal;
27.            System.out.println("Tarik tunai Rp " + nominal + "
berhasil. Saldo saat ini : Rp" + saldo);
28.        } else {
29.            System.out.println("Transaksi Gagal: Saldo tidak
mencukupi. Saldo Anda: Rp" + saldo);
30.        }
31.    }
32.
33.    public boolean cocokDenganNomor(String nomor) {
34.        if (this.nomorRekening.equals(nomor)) {
35.            return true;
36.        } else {
37.            return false;
38.        }
39.    }
40.
```

```

41. public void cekInformasi() {
42.     System.out.println("--- INFO REKENING ---");
43.     System.out.println("No. Rekening : " + nomorRekening);
44.     System.out.println("Nama Pemilik : " + namaPemilik);
45.     System.out.println("Saldo Akhir : " + saldo);
46.     System.out.println("-----");
47. }
48.
49. }

```

Kelas **Rekening** dalam package `Pekan_2` berfungsi sebagai blueprint objek rekening bank, dengan tiga atribut: `nomorRekening` dan `namaPemilik` (String), serta `saldo` (double). Constructor `Rekening(String nomor, String nama, double saldoAwal)` mengisi ketiga atribut tersebut saat objek dibuat, sekaligus mencetak pesan konfirmasi bahwa rekening berhasil dibuat.

Method `setorTunai(double nominal)` menambah saldo jika nominal yang disetor lebih besar dari 0, lalu mencetak saldo terbaru; jika nominal tidak valid (≤ 0), transaksi ditolak dengan pesan error.

Method `tarikTunai(double nominal)` mengurangi saldo, tapi dengan syarat ganda: nominal harus ≤ 10000 **dan** tidak melebihi saldo yang tersedia. Kalau salah satu syarat tidak terpenuhi, transaksi gagal dan pesan menampilkan sisa saldo. (Perlu dicatat, batas 10000 di sini kemungkinan dimaksudkan sebagai limit maksimal penarikan per transaksi, bukan syarat "saldo minimal".)

Method `cocokDenganNomor(String nomor)` membandingkan `nomorRekening` milik objek saat ini dengan nomor yang diinput, lalu mengembalikan nilai `true` atau `false` — berguna saat mencari rekening tertentu di dalam kumpulan objek (misalnya `ArrayList<Rekening>`).

Terakhir, `cekInformasi()` menampilkan seluruh data rekening (nomor, nama, saldo) dalam format yang rapi menggunakan garis pembatas.

b. Class Main

```
1 package Pekan_1;
2
3 import java.util.ArrayList;
4
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         Rekening akunAktif = null; // Objek belum diinisialisasi(null)
10        boolean isRunning = true;
11
12        ArrayList<Rekening> rekeningAktif = new ArrayList<>();
13
14        System.out.println("=== SISTEM PERBANKAN MINI ===");
15
16        while (isRunning) {
17            System.out.println("\nMenu Utama : ");
18            System.out.println("1. Buka Rekening Baru");
19            System.out.println("2. Setor Tunai");
20            System.out.println("3. Tarik Tunai");
21            System.out.println("4. Cek Informasi Rekening");
22            System.out.println("5. Ganti Akun");
23            System.out.print("Pilih Menu : ");
24
25            int pilihan = input.nextInt();
26            input.nextLine(); // Membersihkan buffer enter
27
28            switch (pilihan) {
29                case 1 :
30                    System.out.print("Masukkan No Rekening: ");
31                    String no = input.nextLine();
32                    System.out.print("Masukkan Nama Pemilik : ");
33                    String nama = input.nextLine();
34                    System.out.print("Masukkan Saldo Awal : ");
35                    double saldo = input.nextDouble();
36
37                    // Instansi Objek / Menjalankan Constructor
38                    akunAktif = new Rekening(no, nama, saldo);
39
40                    rekeningAktif.add(akunAktif);
41                    System.out.println("Rekening berhasil ditambahkan dan ditupuk. Total rekening: " + rekeningAktif.size());
42                    break;
43
44                case 2 :
45                    if (akunAktif == null) {
46                        System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening!");
47                    } else {
48                        System.out.print("Masukkan nominal setor : ");
49                        double setor = input.nextDouble();
50                        akunAktif.setorTunai(setor); // Memanggil Behavior / method
51                    }
52                    break;
53
54                case 3 :
55                    if (akunAktif.saldo < 10000) {
56                        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + akunAktif.saldo);
57                    } else {
58                        System.out.print("Masukkan nominal tarik : ");
59                        double tarik = input.nextDouble();
60                        akunAktif.tarikTunai(tarik);
61                    }
62                    break;
63
64                case 4 :
65                    if (akunAktif == null) {
66                        System.out.println("Error : Anda belum membuka rekening!");
67                    } else {
68                        akunAktif.cekInformasi();
69                    }
70                    break;
71
72                case 5 :
73                    if (rekeningAktif.isEmpty()) {
74                        System.out.println("Belum ada rekening yang terdaftar!");
75                    } else {
76                        System.out.print("Masukkan nomor rekening yang ingin diaktifkan : ");
77                        String cariNomor = input.nextLine();
78                        boolean ditemukan = false;
79                        for (Rekening rA : rekeningAktif) {
80                            if (rA.cocokDenganNomor(cariNomor)) {
81                                akunAktif = rA;
82                                ditemukan = true;
83                                System.out.println("Berhasil! Akun aktif sekarang atas nama " + rA.namaPemilik
84                                    + " (No. Rekening: " + rA.nomorRekening + ")");
85                                break;
86                            }
87                        }
88                    }
89                    if (!ditemukan) {
90                        System.out.println("Gagal: Nomor rekening " + cariNomor + " tidak ditemukan.");
91                    }
92                    break;
93
94                case 0 :
95                    isRunning = false;
96                    System.out.println("Sistem ditutup. Terima Kasih!");
97                    break;
98
99                default:
100                    System.out.println("Pilihan tidak valid!");
101            }
102        }
103        input.close();
104    }
105 }
106 }
```

Kelas **Main** dalam package **Pekan_2** berfungsi sebagai program utama yang menjalankan sistem perbankan mini berbasis menu, dengan Scanner input untuk membaca input pengguna, Rekening **akunAktif** yang awalnya bernilai null sebagai penampung rekening yang sedang aktif, boolean **isRunning** sebagai flag untuk mengontrol perulangan menu, dan **ArrayList<Rekening> rekeningAktif** untuk menyimpan seluruh rekening yang

telah dibuat. Selama `isRunning` bernilai `true`, program menampilkan menu pilihan (Buka Rekening Baru, Setor Tunai, Tarik Tunai, Cek Informasi Rekening, Ganti Akun, dan Keluar) lalu memproses pilihan user lewat `switch-case`. Pada **case 1**, program meminta nomor, nama, dan saldo awal, lalu membuat objek baru dengan constructor `Rekening()`, menjadikannya `akunAktif`, dan menambahkannya ke `rekeningAktif`. Pada **case 2**, jika `akunAktif` tidak `null`, program meminta nominal setor lalu memanggil method `setorTunai()`. Pada **case 3**, program mengecek `akunAktif.saldo < 10000` sebelum meminta nominal tarik dan memanggil `tarikTunai()` — namun kondisi ini sebenarnya keliru karena menolak transaksi ketika saldo *kurang* dari 10000, padahal validasi jumlah tarik yang benar sudah ada di dalam method `tarikTunai()` itu sendiri. Pada **case 4**, program memanggil `cekInformasi()` jika `akunAktif` tidak `null`. Pada **case 5**, jika `rekeningAktif` tidak kosong, program meminta nomor rekening yang ingin diaktifkan, lalu melakukan pencarian dengan `for-each` sambil memanggil `cocokDenganNomor()` pada setiap objek; jika cocok, `akunAktif` diganti ke rekening tersebut. Pada **case 0**, `isRunning` diset `false` sehingga loop berhenti dan program menampilkan pesan penutup, sedangkan **default** menangani input di luar pilihan yang tersedia. Di akhir, `input.close()` menutup Scanner untuk melepas resource. Singkatnya, kelas ini adalah pengendali alur program (controller) yang memanfaatkan objek-objek `Rekening` untuk membuat, berpindah, dan bertransaksi pada rekening yang aktif, dengan satu bug logika pada validasi tarik tunai di case 3 yang perlu diperbaiki.

Output :

```
=== SISTEM PERBANKAN MINI ===
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 1
Masukkan No Rekening: 000
Masukkan Nama Pemilik : Fedo
Masukkan Saldo Awal : 100000
Rekening atas nama Fedo berhasil dibuat dengan saldo Rp 100000.0
Rekening berhasil ditambahkan dan ditumpuk. Total rekening: 1
```

```

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 4
--- INFO REKENING ---
No. Rekening : 000
Nama Pemilik : Fedo
Saldo Akhir : 100000.0
-----

```

c. Class Transaksi

```

1. package Pekan_2;
2.
3. public class Transaksi {
4.     String idTransaksi;
5.     String jenis;
6.     double nominal;
7.
8.     // Costructor
9.     public Transaksi(String id, String jenis, double nominal) {
10.         this.idTransaksi = id;
11.         this.jenis = jenis;
12.         this.nominal = nominal;
13.     }
14.
15.     public void cetakDetail() {
16.         System.out.println("ID : " + idTransaksi + " |
17.         Jenis : " + jenis + " | Nominal : Rp" + nominal);
18.     }

```

Kelas **Transaksi** dalam package `Pekan_2` berfungsi sebagai blueprint objek untuk merepresentasikan satu transaksi keuangan, dengan tiga atribut: `idTransaksi` dan `jenis` (String), serta `nominal` (double). Constructor `Transaksi(String id, String jenis, double nominal)` mengisi ketiga atribut tersebut menggunakan kata kunci `this` untuk membedakan atribut objek dengan parameter yang memiliki nama sama (`this.idTransaksi = id`, `this.jenis = jenis`, `this.nominal = nominal`), tanpa mencetak pesan konfirmasi apa pun saat objek dibuat.

Method `cetakDetail()` menampilkan seluruh data transaksi (ID, jenis, dan nominal) dalam satu baris dengan format yang rapi menggunakan tanda pemisah `"|"`.

2. Tugas/Latihan Praktikum

A. Integrasi Rekam Jejak Penarikan

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

a. Class Rekening

```
1. public void tarikTunai(double nominal) {  
2.     if (nominal >= 10000 && nominal <= saldo) {  
3.         saldo -= nominal;  
4.         String idTrx = "TRX-T-" +  
5.         System.currentTimeMillis();  
6.         Transaksi trxBaru = new Transaksi(idTrx,  
7.         "Debit", nominal);  
8.         riwayatTransaksi.add(trxBaru);  
9.         System.out.println("Tarik tunai Rp " +  
10.         nominal + " berhasil. Saldo saat ini : Rp" + saldo);  
11.     } else {  
12.         System.out.println("Transaksi Gagal: Saldo  
13.         tidak mencukupi. Saldo Anda: Rp" + saldo);  
14.     }  
15. }
```

Method tarikTunai(double nominal) berfungsi untuk mengurangi saldo rekening dengan syarat ganda yang harus dipenuhi bersamaan (&&), yaitu nominal penarikan harus kurang dari atau sama dengan 10000 dan tidak boleh melebihi saldo yang tersedia — jika kedua syarat terpenuhi, saldo dikurangi (saldo -= nominal), lalu dibuat objek Transaksi baru dengan jenis "Debit" dan ID unik berbasis waktu (System.currentTimeMillis()), yang kemudian disimpan ke dalam riwayatTransaksi sebagai histori penarikan, dan dicetak pesan sukses beserta saldo terkini. Namun jika salah satu syarat saja tidak terpenuhi (misalnya nominal lebih dari 10000 atau melebihi saldo yang ada), transaksi ditolak, tidak ada objek Transaksi yang dibuat, dan dicetak pesan bahwa saldo tidak mencukupi beserta saldo yang tersisa.

B. Fitur Cetak Mutasi Rekening

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini"
- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

a. Class Rekening

```
1. public void cetakMutasi() {  
2.     if (riwayatTransaksi.isEmpty()) {  
3.         System.out.println("Belum ada transaksi pada rekening  
4.         ini");  
5.     } else {  
6.         for (Transaksi trx : riwayatTransaksi) {  
7.             trx.cetakDetail();  
8.         }  
9.     }
```

Method cetakMutasi() berfungsi untuk menampilkan seluruh riwayat transaksi yang pernah terjadi pada suatu rekening. Pertama, method ini mengecek apakah riwayatTransaksi (list ArrayList<Transaksi>) kosong dengan isEmpty() — jika benar kosong, dicetak pesan bahwa belum ada transaksi pada rekening tersebut. Namun jika tidak kosong, program menggunakan perulangan for-each untuk mengiterasi setiap objek Transaksi trx di dalam riwayatTransaksi, dan pada setiap iterasi memanggil method trx.cetakDetail() milik objek tersebut, yang akan menampilkan ID, jenis, dan nominal dari masing-masing transaksi secara berurutan.

b. Class Main

```
1. case 6 :  
2.         akunAktif.cetakMutasi();  
3.         break;
```

Blok case 6 ini menambahkan opsi menu baru pada switch-case di kelas Main, yang berfungsi untuk mencetak mutasi (riwayat transaksi) dari rekening yang sedang aktif. Ketika user memilih menu 6, program langsung memanggil `akunAktif.cetakMutasi()`, yang akan menampilkan seluruh histori transaksi (setor dan tarik) milik `akunAktif` — atau pesan "belum ada transaksi" jika riwayatnya masih kosong. Setelah itu, `break` menghentikan eksekusi switch agar tidak lanjut memeriksa case-case berikutnya.

Output :

```
Menu Utama :  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi  
Pilih Menu : 6  
ID : TRX-S-1789664955328 | Jenis : Kredit | Nominal : Rp100000.0  
ID : TRX-T-1789664961114 | Jenis : Debit | Nominal : Rp100000.0
```